

Report on the Simultaneous Translation from Czech to English

Jan Mitka^{1†} and Ori Moscovitz¹ and David Ruda¹ and Pavel Šindelář¹

¹Charles University, Faculty of Mathematics and Physics

[†]Team Leader

Abstract

This work focuses on the text-to-text simultaneous translation from Czech to English, following the [IWSLT 2025 guidelines for the LLM track](#). We investigate two strategies: Local Agreement and AlignAtt, to balance translation quality and responsiveness. Our experiments reveal a trade-off between translation quality and latency, with fine-tuning significantly improving BLEU scores at the cost of increased lag. All code, models, and data are available at <https://github.com/MagicSenten/Simul-Translation>.

1 Introduction

Simultaneous translation presents a fundamental challenge in natural language processing, requiring systems to produce accurate translations while processing incomplete source sentences in real-time. The difficulty is that, with fixed-output simultaneous translation, new information may alter the best translation hypothesis, but the already-produced text is final and cannot be changed. Our work focuses specifically on the text-to-text part of simultaneous translation from Czech to English, following the guidelines of the Simultaneous Speech Translation task at IWSLT 2025 in the LLM track.

Previous approaches have explored various strategies, including wait-k policies (Papi et al., 2023a), prefix-to-prefix training (Ma et al., 2019), and attention-based alignment mechanisms for speech-to-text (Papi et al., 2023b).

In our research, we explore two primary algorithmic techniques: Local Agreement (Liu et al., 2020), which delays output until sufficient agreement is reached over multiple decoding steps, and AlignAtt (Papi et al., 2023b), which leverages intermediate encoder representations to improve source–target alignment during translation. We apply both methods first to an off-the-shelf NMT model and then to that model after fine-tuning it on our parallel

corpus, to assess how in-domain adaptation affects simultaneous translation performance. We also assess an off-the-shelf large language model for zero-shot translation.

2 Chosen Strategies

Because the committed output can never be changed, we always force the model to take it into account by forcing the already committed output on the decoder.

2.1 Local Agreement

Local Agreement-n (Polák et al., 2022) is based on Local Agreement (Liu et al., 2020). It solves the issue of outputting uncertain translations in two ways. When we have fewer than n input chunks, we do not commit anything. This ensures that the model has enough context to work with.

After we have read the first n input chunks, we use Local Agreement. This method works by assuming that if a part of a partial translation does not change with a few new inputs, it is more likely not to change even after many new inputs. We compare the new output tokens (discarding those that were already committed at time c) $W^{(c)}$ with the same value at the previous step $W^{(c-1)}$. We determine the text to output as the PREFIX of $W^{(c)}$ supported by $W^{(c-1)}$, defined as

$$\text{PREFIX}(W^{(c)}) = \text{LCP}(\text{DISCARD}(W^{(c-1)}), W^{(c)})$$

where $\text{DISCARD}(W)$ indicates tokens in W but not in $\text{PREFIX}(W)$ and $\text{LCP}(\cdot)$ indicates the longest common prefix of two lists.

2.2 ALIGNATT

ALIGNATT (Papi et al., 2023b) (an evolution of EDATT (Papi et al., 2023a)) is a method that uses the fact that when a transformer translation model emits a token while attending only to the information read from the input last, such an emission is likely to require more context.

Given an attention matrix A_c

$$A_c(Q, K, V) = \text{softmax}(QK^T \sqrt{d_k})V,$$

where $Q, K, V \in R^{n \times d_k}$, we can define Align_i as

$$\text{Align}_i = \arg \max_j A_c(x, y_i).$$

We emit tokens until we find a token that mostly attends to one of the last f read inputs. That is while

$$\text{Align}_i \notin \{n - f + 1, \dots, n\}.$$

In the case of the NMT backbone, we modify ALIGNATT to work with text by simply using the encoder-decoder attentions resulting from the text inputs.

In the case of EuroLLM, we use the self-attention matrix and only consider the positions in the prompt when checking if an output attends to the last f read inputs.

3 Evaluation Metrics

Evaluating simultaneous translation requires a focus on translation quality and real-time latency. We introduce **SimuEval**, a Python-based evaluation for our simultaneous translation models. The evaluation includes three metrics - **BLEU** and **Translation Error Rate (TER)** for quality evaluation and **Average Lagging (AL)** for latency measurement.

3.1 Quality Metrics

3.1.1 Bilingual Evaluation Understudy (BLEU)

We compute the corpus-level BLEU score using the sacreBLEU library (Post, 2018). The BLEU score is evaluated by comparing n-gram overlap between a sequences of hypotheses strings and a sequences of references (single or multiple references). SacreBLEU computes the results for unigrams, bigrams, trigrams, and four-grams, aggregates the results by geometric mean, followed by applying a brevity penalty. In our case, we have one reference for each hypothesis.

While BLEU is a standard metric for evaluating translation quality, its scores in simultaneous translation are typically lower than in offline translation. In offline settings, the model has access to the full source sentence, enabling coherent and fully contextualized output. Simultaneous translation, on the other hand, operates with partial input, forcing the model to commit with limited context.

This often results in chunked output and less accurate reordering, both of which BLEU penalizes harshly. Therefore, lower BLEU scores in simultaneous translation are expected and reflect the latency–quality trade-off inherent to the task, rather than serving as a direct indicator of poor translation quality.

3.1.2 Translation Error Rate (TER)

We calculated TER using word error rate (WER) for sentence-level accuracy evaluation. The TER is computed for each sentence pair and then averaged. (Jitsi, 2019)

$$\text{TER} = \frac{S + D + I}{N} \quad (1)$$

Where S , D , I , and N are the number of substitutions, deletions, insertions, and total numbers of words in the reference text, respectively.

3.2 Latency Metric

3.2.1 Average Lagging (AL)

To measure latency, we utilized Average Lagging (AL), a widely adopted metric first proposed by Ma et al. (2019) (Ma et al., 2020). AL measures how much a simultaneous translation model "lags" behind an ideal agent that translates at a constant rate.

In this work, as it is a **text-to-text** translation, we measure the average lagging in words. The average Czech speaker speaks four words per second¹, so our results can be divided by four to obtain the average lagging in seconds rather than in words.

The Average Lagging (AL) metric is computed as:

$$AL = \frac{1}{\tau} \sum_{i=1}^{\tau} \left(D_i - (i - 1) \frac{|X|}{|Y|} \right) \quad (2)$$

Given X as the source, Y as the target, and D as the delays, the output is the lagging score for a single sentence. We then average the result over all sentences. τ is the "cut-off" step, which is defined as the decoding step when source sentence finishes:

$$\tau = \arg \min_i (D_i = |X|) \quad (3)$$

¹The average Czech speech rate is approximately 3–5 words per second.

4 Models used

4.1 NMT backbone

For the encoder-decoder backbone, we decided to use the version of OPUS-MT (Tiedemann and Thottingal, 2020; Tiedemann, 2023) trained for Czech-to-English translation². The project that the model comes from trained the same architecture on many different translation datasets, and Czech, being a relatively high-resource language, is among them.

The model architecture is the basic transformer encoder-decoder architecture with six layers and eight-head attention. The sizes were the same as those of the original transformer paper (Vaswani, 2017), which means that the base size was 512 and the expanded size in the FF block was 2048.

We used the implementation in the transformers library because it allows us to easily access the attention values.

4.2 EuroLLM

For the LLM experiments, we decided to use the 1.7B version of EuroLLM (Martins)³. The model had two variants, the smaller 1.7 billion parameter version and the later enlarged 9 billion parameter version. The model was trained on a mixture of 50% English data, 5% code data, and 45% of other language data. The proportions of the other language data are based on their quantities; this data also consists of 20% of parallel texts.

It is a decoder-only model. As with all LLMs, it needed to be fine-tuned for instruction following. This was achieved by supervised fine-tuning on an instruction following dataset.

In order to use it for simultaneous translation, the model was prompted to translate using the prompt in Figure 1.

5 Finetuning

To improve the performance of the NMT backbone in simultaneous translation, we finetuned the model on our created dataset of Czech-English sentence prefix pairs.

5.1 Prefix Training Dataset

To simulate the simultaneous-translation task, we convert each full parallel sentence pair into a small set of “prefix” pairs. From each sentence pair, we

²<https://huggingface.co/Helsinki-NLP/opus-mt-cs-en>

³<https://huggingface.co/utter-project/EuroLLM-1.7B-Instruct>

```
<|im_start|>system
You are simultaneous interpreter
from Czech to English, you translate
incomplete sentences, please make
sure you only translate what is
explicitly stated in the input segment.
<|im_end|><|im_start|>user
Translate the following Czech
source text to English.
Czech: {input_text}
English: <|im_end|>
<|im_start|>assistant
```

Figure 1: The prompt used with EuroLLM 1.7B in our experiments.

```
{"pref_source": "Já",
"pref_target": "I'm"}
{"pref_source": "Já jsem Roderick.",
"pref_target": "I'm Roderick."}
{"pref_source": "Všechny oběti",
"pref_target": "All the victims"}
{"pref_source": "Všechny",
"pref_target": "All the"}
```

Figure 2: Examples of prefix-sentence pairs.

first identify the shorter sentence and then generate prefixes by progressively adding words from the longer sentence in proportion to their relative lengths (measured in characters). This yields many possible prefix combinations per pair, but to limit overfitting we randomly sample and keep just two prefixes for each original pair. Check fig. 2 for examples. For step-by-step instructions and code, see Data_preparation/README.md in our GitHub repository.

5.2 Finetuning Details

The finetuning dataset contained 1.7 million training examples and 5 thousand validation examples. We ensured that the training and validation data contained prefixes from distinct original sentences to prevent data leakage. We used NVIDIA H100 94GB to finetune the model.

Before finetuning, the NMT backbone achieved 55.80 BLEU on the validation set. After finetuning, it achieved **64.45 BLEU** on the validation set. The model is available on the Hugging Face Hub⁴, where you can find specific hyperparameter settings. The exact steps used to finetune the model are described in Finetuning/README.md inside our GitHub repository.

⁴<https://huggingface.co/davidruda/opus-mt-cs-en-Prefix-Finetuned>

	BLEU	TER	AL
NMT Finetuned	17.62	0.76	9.06
LA NMT Baseline	16.53	0.78	9.11
AlignAtt NMT Finetuned	16.43	0.73	9.36
LA NMT Finetuned	16.07	0.74	9.53
AlignAtt NMT Baseline	14.30	0.78	8.89
NMT Baseline	13.09	0.83	7.82
LA EuroLLM	11.33	0.79	9.33

Table 1: Only the best overall performance for each model/approach is listed. 'TER' stands for Translation Error Rate. 'AL' stands for Average Lagging.

6 Experiment Results

All experiments were conducted on a subset of 300 sentence pairs randomly selected from the IWSLT2024 Czech development set. To facilitate reproducibility, you can find the subset in our GitHub repository under `Data_preparation/cleaned_eval_dataset.jsonl`.

6.1 Overall Results

Looking at Table 1, the fine-tuned NMT backbone achieved the highest BLEU (17.62) but at a slightly increased latency (AL = 9.06). In contrast, the bare NMT baseline is the fastest (lowest AL = 7.82) yet yields only 13.09 BLEU. Applying Local Agreement or AlignAtt to the fine-tuned model reduces BLEU (16.43 and 16.07, respectively) compared to the vanilla fine-tuned backbone. The AlignAtt-finetuned variant attains the lowest TER (0.73), showing its strength at reducing literal errors, whereas the EuroLLM + Local Agreement model scores worst on BLEU (11.33), likely due to hallucinations. Overall, there is a clear quality-latency trade-off: fine-tuning improves translation quality at the expense of speed.

6.2 NMT Baseline and Fine-Tuned Experiments

We first ran two sets of experiments on the vanilla NMT backbone and on its fine-tuned variant, performing a grid search over beam size (#B) and the number of source tokens to buffer ("wait at begin") before starting to translate. From this search, we took the best BLEU settings into our AlignAtt experiments, so there we could focus exclusively on its novel parameters.

As Table 2 shows, increasing the beam count from 1→3 consistently reduces BLEU for the backbone

across all initial-delay settings. Table 3 for the fine-tuned model exhibits the same overall trend, the only exception being the third column where the BLEU score increases. By contrast, longer delays predictably raise Average Lagging but lower TER in both models, though the TER drop is more pronounced in the NMT baseline.

6.3 AlignAtt Experiments

As shown in Table 4, for the vanilla backbone, AlignAtt fed with encoders' layer 2 yields the best BLEU scores for all of the frame sizes. The best setting increases the BLEU score by 1.44 from the "starting point", which is the best result of the NMT backbone (beam=1, wait=2) from the Table 2. Attending to the 4th or 3rd encoders' layer drags BLEU down. The translation error rate is essentially constant; the same applies to the average lag. Table 5 confirms the same pattern we saw before: tapping the encoder's second layer delivers the best BLEU scores. However, even this best AlignAtt configuration still falls short of our fine-tuned NMT baseline (beam=2, wait=3) from Table 3. More strikingly, feeding AlignAtt from layers 3 or 4 slashes BLEU by nearly ten points, indicating a dramatic collapse in translation quality.

6.4 Local Agreement Experiments

Table 6 reports BLEU, Average Lagging (AL), and TER for beam sizes 1–3 and initial delays of 1–3 tokens for the Local Agreement on the NMT baseline. Increasing the delay consistently boosts BLEU and lowers TER—albeit at the cost of higher AL—while larger beams uniformly reduce BLEU and worsen both AL and TER. Thus, a single beam with moderate buffering delivers the best balance of accuracy, latency, and error rate.

Table 7 demonstrates that, after fine-tuning and compared to our raw model (beam=1, wait=1; BLEU = 17.58), longer initial buffering steadily raises BLEU from 15.14→16.13 and reduces TER from 0.77→0.74, at the cost of a small AL increase (9.40→9.60). Beam size has only a marginal and inconsistent effect on BLEU and TER. Overall, even the best local-agreement configuration underperforms the untweaked fine-tuned backbone by 1.45 BLEU points.

Table 8 compares the EuroLLM's performance under our same beam-and-delay grid search. Its top BLEU of 11.78 remains well below our NMT baselines, and TER stays essentially unchanged across

#B	1 (Wait At Begin)	2 (Wait At Begin)	3 (Wait At Begin)
1	BLEU: 11.43, AL: 7.75 , TER: 0.87	BLEU: 13.09 , AL: 7.82, TER: 0.83	BLEU: 12.40, AL: 8.20, TER: 0.83
2	BLEU: 10.44, AL: 7.88, TER: 0.89	BLEU: 11.09, AL: 8.08, TER: 0.86	BLEU: 11.29, AL: 8.40, TER: 0.82
3	BLEU: 9.99, AL: 7.95, TER: 0.90	BLEU: 11.18, AL: 8.14, TER: 0.87	BLEU: 11.64, AL: 8.41, TER: 0.83

Table 2: NMT baseline experiment results. 'TER' stands for Translation Error Rate. 'AL' stands for Average Lagging.

#B	1 (Wait At Begin)	2 (Wait At Begin)	3 (Wait At Begin)
1	BLEU: 17.01, AL: 8.71, TER: 0.80	BLEU: 17.21, AL: 8.88, TER: 0.78	BLEU: 17.04, AL: 9.13, TER: 0.77
2	BLEU: 16.79, AL: 8.72, TER: 0.78	BLEU: 16.82, AL: 8.92, TER: 0.78	BLEU: 17.62 , AL: 9.06, TER: 0.76
3	BLEU: 16.84, AL: 8.70 , TER: 0.79	BLEU: 16.90, AL: 8.93, TER: 0.78	BLEU: 17.53, AL: 9.14, TER: 0.76

Table 3: NMT finetuned experiment results. 'TER' stands for Translation Error Rate. 'AL' stands for Average Lagging.

FS	4. Layer	3. Layer	2. Layer
1	BLEU: 11.53, AL: 8.90, TER: 0.78	BLEU: 11.19, AL: 8.70 , TER: 0.78	BLEU: 13.75, AL: 8.80, TER: 0.79
2	BLEU: 11.28, AL: 8.91, TER: 0.78	BLEU: 11.09, AL: 8.72, TER: 0.78	BLEU: 14.30, AL: 8.89, TER: 0.78
3	BLEU: 11.10, AL: 8.94, TER: 0.78	BLEU: 10.92, AL: 8.73, TER: 0.78	BLEU: 14.53 , AL: 9.06, TER: 0.77

Table 4: AlignAtt NMT baseline experiment results. Starting from the setting with 13.09 BLEU, 7.82 AL, and 0.83 TER, but that setting uses the last layer of the NMT backbone. 'FS' stands for Frame Size. 'TER' stands for Translation Error Rate. 'AL' stands for Average Lagging.

FS	4. Layer	3. Layer	2. Layer
1	BLEU: 8.34, AL: 9.06, TER: 0.78	BLEU: 8.92, AL: 9.06, TER: 0.77	BLEU: 15.19, AL: 9.37, TER: 0.74
2	BLEU: 8.24, AL: 9.05 , TER: 0.78	BLEU: 8.80, AL: 9.08, TER: 0.77	BLEU: 15.27 , AL: 9.39, TER: 0.74
3	BLEU: 8.07, AL: 9.08, TER: 0.78	BLEU: 8.87, AL: 9.13, TER: 0.77	BLEU: 15.15, AL: 9.36, TER: 0.74

Table 5: AlignAtt NMT finetuned experiment results. Starting from the setting with 17.62 BLEU, 9.06 AL, and 0.76 TER, but that setting uses the last layer of the finetuned model. 'FS' stands for Frame Size. 'TER' stands for Translation Error Rate. 'AL' stands for Average Lagging.

#B	1 (Wait At Begin)	2 (Wait At Begin)	3 (Wait At Begin)
1	BLEU: 11.43, AL: 7.75 , TER: 0.87	BLEU: 13.09 , AL: 7.82, TER: 0.83	BLEU: 12.40, AL: 8.20, TER: 0.83
2	BLEU: 10.44, AL: 7.88, TER: 0.89	BLEU: 11.09, AL: 8.08, TER: 0.86	BLEU: 11.29, AL: 8.40, TER: 0.82
3	BLEU: 9.99, AL: 7.95, TER: 0.90	BLEU: 11.18, AL: 8.14, TER: 0.87	BLEU: 11.64, AL: 8.41, TER: 0.83

Table 6: Local Agreement NMT Baseline

#B	1 (Wait At Begin)	2 (Wait At Begin)	3 (Wait At Begin)
1	BLEU: 15.14, AL: 9.40, TER: 0.77	BLEU: 15.40, AL: 9.44, TER: 0.76	BLEU: 16.13 , AL: 9.60, TER: 0.75
2	BLEU: 15.39, AL: 9.37, TER: 0.77	BLEU: 15.67, AL: 9.50, TER: 0.76	BLEU: 15.89, AL: 9.53, TER: 0.75
3	BLEU: 15.34, AL: 9.34 , TER: 0.77	BLEU: 15.15, AL: 9.47, TER: 0.76	BLEU: 16.07, AL: 9.53, TER: 0.74

Table 7: Local Agreement NMT Finetuned

#B	1 (Wait At Begin)	2 (Wait At Begin)	3 (Wait At Begin)
1	BLEU: 11.33, AL: 9.33, TER: 0.79	BLEU: 11.62, AL: 9.73, TER: 0.79	BLEU: 11.02, AL: 10.25, TER: 0.79
2	BLEU: 11.34, AL: 9.29 , TER: 0.80	BLEU: 11.78 , AL: 9.75, TER: 0.79	BLEU: 11.44, AL: 10.23, TER: 0.80
3	BLEU: 11.14, AL: 9.33, TER: 0.80	BLEU: 11.58, AL: 9.73, TER: 0.79	BLEU: 11.55, AL: 10.22, TER: 0.79

Table 8: Local Agreement EuroLLM

settings. As before, increasing the “wait at begin” delay predictably raises Average Lagging.

7 Conclusion

Our experiments reveal a trade-off between translation quality and latency. Fine-tuning the NMT model on a prefix-based dataset substantially improved translation quality, yielding the highest BLEU score of 17.62. This gain was directly offset by higher latency, with Average Lagging increasing from 7.82 to 9.06. Furthermore, applying Local Agreement or AlignAtt failed to improve the fine-tuned model’s top BLEU score, while boosting the scores of the non-fine-tuned model. This suggests that the classical fine-tuning process optimizes the weights in a way that conflicts with the mechanisms of these real-time strategies.

In future work, it might be worthwhile to find ways of fine-tuning that do not jeopardize the efficiency of these inference strategies. Alternatively, the strategies themselves could be adapted to work with prefix-finetuned models.

8 Acknowledgments

Computational resources were provided by the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth and Sports of the Czech Republic.

Thanks to Dominik Macháček and Peter Polák of Charles University for assisting with our project.

References

- Jitsi. 2019. [jiwer: A simple and fast python package to evaluate an automatic speech recognition system](#).
- Danni Liu, Gerasimos Spanakis, and Jan Niehues. 2020. [Low-latency sequence-to-sequence speech recognition and translation by partial hypothesis selection](#). pages 3620–3624.
- Xutai Ma, Mohammad Javad Dousti, Changan Wang, Jiatao Gu, and Juan Pino. 2020. [Simuleval: An evaluation toolkit for simultaneous translation](#). pages 144–150.
- Xutai Ma, Liang Huang, and Caiming Xiong. 2019. [STACL: Simultaneous translation with implicit anticipation and controllable latency using prefix-to-prefix framework](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3025–3036, Florence, Italy. Association for Computational Linguistics.
- Pedro Henrique *et al.* Martins. [Eurollm: Multilingual language models for europe](#), 2024.
- Sara Papi, Matteo Negri, and Marco Turchi. 2023a. [Attention as a guide for simultaneous speech translation](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13340–13356, Toronto, Canada. Association for Computational Linguistics.
- Sara Papi, Marco Turchi, and Matteo Negri. 2023b. [Alignatt: Using attention-based audio-translation alignments as a guide for simultaneous speech translation](#). In *Interspeech 2023*, pages 3974–3978.
- Peter Polák, Ngoc-Quan Pham, Tuan Nam Nguyen, Danni Liu, Carlos Mullov, Jan Niehues, Ondřej Bojar, and Alexander Waibel. 2022. [CUNI-KIT system for simultaneous speech translation task at IWSLT 2022](#). In *Proceedings of the 19th International Conference on Spoken Language Translation (IWSLT 2022)*, pages 277–285, Dublin, Ireland (in-person and online). Association for Computational Linguistics.
- Matt Post. 2018. [A call for clarity in reporting bleu scores](#). pages 186–191.
- Jörg Tiedemann and Santhosh Thottingal. 2020. OPUS-MT — Building open translation services for the World. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation (EAMT)*, Lisbon, Portugal.
- Jörg *et al.* Tiedemann. 2023. [Democratizing neural machine translation with OPUS-MT](#). *Language Resources and Evaluation*, (58):713–755.
- Ashish *et al.* Vaswani. 2017. [Attention is all you need](#). In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17*, page 6000–6010, Red Hook, NY, USA. Curran Associates Inc.

9 Appendix

A Overview of individual contributions of team members

A.1 Jan Mitka

As the team leader for this project, Jan coordinated team efforts, managed task allocation, and drove the project's progress. Jan's technical contributions included preparing the prefix training dataset for fine-tuning, creating the evaluation split for benchmarking, and collaborating with Pavel to define the parameter space for the grid search. Jan also authored the [Experiment Results](#) section, detailing the outcomes of the work.

A.2 Ori Moscovitz

Ori developed and implemented the [evaluation framework](#), consisting of BLEU, translation error rate and average lagging and ensured integration of the evaluation module with the main system during development. In addition, she contributed to the written documentation by authoring the [Abstract](#), [Introduction](#), and [Evaluation Metrics](#) sections, as well as the top-level project documentation, including the [README file](#) and consolidated requirements.

A.3 David Ruda

David handled the model prefix fine-tuning, helping to integrate our project with the Metacentrum computation cluster. He managed hyperparameter selection and validation set testing for prefix translation. He described the method used and the results on the validation set in detail in the [Finetuning](#) section.

A.4 Pavel Šindelář

Pavel handled the Local Agreement and AlignAtt strategies and inference implementations. He managed the strategy parameter tuning experiments. During the formulation of the paper, he authored the [Chosen Strategies](#) section describing the strategies implemented. He also authored the [Models used](#) section describing the models we decided to use in the experiments.